SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I **R.JAGAN-192321102** that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: R. Jagan

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWERS:

1.Doctor Shift Assignment (CSP and Backtracking)

1. Problem formulation as a CSP

- **Variables:** D1, D2, D3 (each doctor).
- **Domain:** {Morning (M), Afternoon (A), Night (N)}.
- **Constraints:** C1: D1 ≠ N. C2: D3 ≠ M. C3: shift(D2) < shift(D3) with M < A < N. C4: all-different (one doctor per shift, each doctor exactly one shift).

2. Node consistency (unary constraints applied first)

Variable	Initial domain	After C1 and C2
D1	{M, A, N}	{M, A} (C1 removes N)
D2	{M, A, N}	{M, A, N}
D3	{M, A, N}	{A, N} (C2 removes M)

3. Backtracking search (variable order D1, D2, D3; value order M, A, N)

Step	Assignment tried	Constraint check	Result
1	D1 = M	C1: M ≠ N satisfied	Accept
2	D1 = M, D2 = M	C4: Morning already taken by D1	Violated - backtrack
3	D1 = M, D2 = A	C4 satisfied; C3 pending (D3 not yet assigned)	Accept
4	D2 = A, D3 = M	C2 violated (D3 cannot work Morning); C4 violated	Backtrack
5	D2 = A, D3 = A	C4 violated (Afternoon taken by D2)	Backtrack
6	D2 = A, D3 = N	C2 ok; C4 ok; C3: A < N satisfied; C1 ok	All constraints satisfied - SOLUTION

Search tree summary: Root → D1=M → D2=M (fail) | D2=A → D3=M (fail) | D3=A (fail) | D3=N (goal).

4. Final valid schedule

Morning	**Afternoon**	**Night**
D1	D2	D3

Verification: D1 is not on Night (C1); D3 is not on Morning (C2); D2 (Afternoon) comes before D3 (Night) (C3); every shift has exactly one doctor and every doctor exactly one shift (C4).

Note: If the search continues after the first solution, a second valid schedule exists: D2 = Morning, D1 = Afternoon, D3 = Night. Backtracking returns the first solution found, D1-M, D2-A, D3-N.

2.Robot in a 5x5 Grid (BFS)

1. Problem set-up

The question does not include the grid figure, so the following 5x5 grid is assumed. Cells are written (row, column), counted from 0. S = (0,0) is the start, G = (4,4) is the goal, and # marks an obstacle.

	c0	**c1**	**c2**	**c3**	**c4**
r0	S	.	.	.	.
r1	.	#	.	#	.
r2	.	.	.	#	.
r3	.	#	.	.	.
r4	.	.	.	.	G

- **State:** robot position (r, c). **Actions:** Up, Down, Left, Right (expansion order used: U, D, L, R).
- **Step cost:** 1 per move. **Goal test:** position = (4,4). **Heuristic:** h(n) = |r - 4| + |c - 4| (Manhattan distance), shown for reference.
- **Note on the algorithm:** Plain BFS uses a FIFO queue, which is a priority queue whose priority is the depth g(n). Because every move costs 1, BFS is optimal. The queue column below shows node:g, ordered by priority. The Manhattan heuristic is not needed by BFS; it would only be used if A* were applied.

2. Step-by-step node expansion and queue updates

Step	**Expanded**	**g**	**h**	**Children added**	**Queue (node:g) after step**
1	(0,0)	0	8	(1,0), (0,1)	(1,0):1 (0,1):1
2	(1,0)	1	7	(2,0)	(0,1):1 (2,0):2
3	(0,1)	1	7	(0,2)	(2,0):2 (0,2):2
4	(2,0)	2	6	(3,0), (2,1)	(0,2):2 (3,0):3 (2,1):3
5	(0,2)	2	6	(1,2), (0,3)	(3,0):3 (2,1):3 (1,2):3 (0,3):3
6	(3,0)	3	5	(4,0)	(2,1):3 (1,2):3 (0,3):3 (4,0):4

Step	Expanded	g	h	Children added	Queue (node:g) after step
7	(2,1)	3	5	(2,2)	(1,2):3 (0,3):3 (4,0):4 (2,2):4
8	(1,2)	3	5	none (dead end / already visited)	(0,3):3 (4,0):4 (2,2):4
9	(0,3)	3	5	(0,4)	(4,0):4 (2,2):4 (0,4):4
10	(4,0)	4	4	(4,1)	(2,2):4 (0,4):4 (4,1):5
11	(2,2)	4	4	(3,2)	(0,4):4 (4,1):5 (3,2):5
12	(0,4)	4	4	(1,4)	(4,1):5 (3,2):5 (1,4):5
13	(4,1)	5	3	(4,2)	(3,2):5 (1,4):5 (4,2):6
14	(3,2)	5	3	(3,3)	(1,4):5 (4,2):6 (3,3):6
15	(1,4)	5	3	(2,4)	(4,2):6 (3,3):6 (2,4):6
16	(4,2)	6	2	(4,3)	(3,3):6 (2,4):6 (4,3):7
17	(3,3)	6	2	(3,4)	(2,4):6 (4,3):7 (3,4):7
18	(2,4)	6	2	none (dead end / already visited)	(4,3):7 (3,4):7
19	(4,3)	7	1	(4,4)	(3,4):7 (4,4):8
20	(3,4)	7	1	none (dead end / already visited)	(4,4):8
21	(4,4)	8	0	Goal reached	-

3. Optimal path and cost

Path (found by tracing parent pointers from G): (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Moves: Down, Down, Down, Down, Right, Right, Right, Right.

Optimal cost = 8 (8 moves x 1). Nodes expanded: 21.

Why optimal: BFS expands nodes in non-decreasing depth, and all step costs are equal, so the first time G is reached is via a shortest path. The Manhattan lower bound from S to G is also 8, so no path can be cheaper. Other paths of cost 8 also exist, for example along row 0 and then down column 4.

3.Autonomous Rescue Robot

a) Analysis of constraints and their effect on decision-making (2.5 marks)

Constraint	Effect on decision-making
Moves: Up, Down, Left, Right	Branching factor is at most 4. The robot never moves diagonally, so the state space is small and discrete.
Blocked cells (obstacles)	The robot must remove blocked cells from its action list. A path that looks short may be impossible, so it needs backtracking or replanning.
Limited sensing (adjacent cells only)	The robot cannot see the whole map. It decides using a partial, changing map, so it cannot plan the full route in advance.
Move cost 1, risky cell +2 (cost 3)	Path length alone is not enough. A longer path through safe cells can be cheaper than a shorter risky one, so cost-aware search is needed.
Minimise cost with safe navigation	Optimality (least total cost) and safety must be balanced. Risky cells are allowed only when the saving justifies them.
Dynamic knowledge update (online search)	The robot acts and learns at the same time. Each new observation changes the map and may force a new plan.

Priority order: (1) never enter blocked cells (hard constraint); (2) keep sensing and updating the map (needed to act at all); (3) minimise cost including risk penalties (optimisation goal).

b) Solution approach: Online Uniform Cost Search with replanning (3 marks)

Uniform Cost Search (UCS) is used because step costs differ (1 or 3). It is combined with an online agent loop because the map is only partly known.

1. Initialise the knowledge map: the start S, the goal G and the known cells. Cells not yet seen are treated as free (optimistic assumption).
2. Sense: observe the four adjacent cells and mark each as free, blocked or risky.
3. Plan: run UCS on the current known map from the robot's cell to G. The frontier is a priority queue ordered by path cost g(n). Moving into a safe cell costs 1, and into a risky cell costs 1 + 2 = 3.

4. Act: take only the first move of the best plan (or the next move along it) to the neighbouring cell.
5. Update: sense again at the new cell and update the map. If the new information makes the current plan invalid or more costly (new obstacle or risky cell), discard the plan and replan with UCS from the current cell.
6. Mark visited cells and their costs so the robot does not loop (the same idea as LRTA*, which stores learned cost estimates).
7. Repeat steps 2 to 5 until the robot reaches G. If every route is blocked, report that the survivor cannot be reached.

Illustration of cost comparison: Route X has 4 moves, of which 3 are safe (3 x 1 = 3) and 1 enters a risky cell (1 + 2 = 3), so its cost is 3 + 3 = 6. Route Y has 6 safe moves: cost = 6. Both cost 6, so the safer route Y is preferred as the tie-break. If route X had 2 risky cells its cost would be 8, and Y is clearly better.

c) Application of AI concepts (2.5 marks)

Intelligent agent (PEAS):

Component	**Description**
Performance measure	Reach the survivor with minimum total cost, avoid risky zones, stay safe.
Environment	Disaster-hit building as a grid of rooms with obstacles and risky zones.
Actuators	Wheels or legs for Up, Down, Left, Right movement.
Sensors	Adjacent-cell sensors (obstacle, hazard detection) and position sensor.

Agent type: A goal-based and utility-based agent. It keeps an internal model (the updated map), has a goal (reach G), and uses cost as utility to choose among actions.

Rationality: The robot is rational because, given what it has perceived so far, it selects the action expected to maximise its performance measure (lowest total cost with safety). Rationality here means doing the best with the information available, not knowing everything. As new percepts arrive, it revises its choice.

Environment type:

Property	Classification	Reason
Observability	Partially observable	Only adjacent cells are sensed.
Agents	Single agent	Only the robot acts (survivor is passive).
Determinism	Deterministic (assumed)	A move goes where intended; uncertainty comes from missing knowledge.
Episodic / Sequential	Sequential	Each move affects later costs and options.
Static / Dynamic	Static map, dynamic knowledge	Layout is fixed, but the robot's knowledge changes. Collapses could make it dynamic.
Discrete / Continuous	Discrete	Grid cells and four fixed actions.
Known / Unknown	Partly unknown	The map is discovered while exploring (online search).

d) Justification of the chosen approach (2 marks)

- **Cost-aware:** BFS ignores the extra risk cost and Greedy search ignores path cost, so neither guarantees minimum total cost. UCS is optimal for non-uniform positive step costs and is complete.
- **Online:** An offline planner needs the full map beforehand. With limited sensing, the robot must interleave planning and acting, so an online agent with replanning is the right fit.
- **Safe and robust:** Risk is built into the cost function, so the robot avoids hazards unless the saving justifies it. Replanning after each observation handles newly found obstacles.

- **Simple and reliable:** No domain-specific heuristic is required. If a reliable admissible heuristic (Manhattan distance times the minimum step cost of 1) is available, UCS can be upgraded to A* to reduce nodes expanded, while keeping optimality on the known map.
- **Limitation:** In a partially observable world, no algorithm can guarantee a globally optimal route in advance. The approach gives the best cost based on current knowledge and is optimal when the map is fully discovered.

Conclusion: Online UCS with replanning meets all constraints (obstacles, limited sensing, risky-zone cost, dynamic knowledge) and gives a rational, cost-minimising and safe rescue route.